COMPUTACIÓ PARAL·LELA I MASSIVA - 1.2

Nom: Jordà Coca Rodríguez
Curs: 2025 - 2026

Índex

Enunciat

Algorisme de multiplicació de matrius disperses. Tres versions: A dispersa, A i B disperses i A,B i C disperses.

Temps d'execució seqüencial amb les opcions "-O3" per N=8000

a Teen -> 16.2 segons,

a Orca -> 48.5 segons

- Es programarà amb OpenMP
- Mitja harmònica mínima de l'speedup de totes les execions a assolir: a Teen: 3.5, a Orca: 5
- La versió paral·lela ha de donar el mateix resultat que la seqüencial. (Cal comprovar els valors de sortida)
- Cal executar el codi, forçant la creació de threads de 2 fins a 32 per teen, i de 2 fins a 128 per orca amb potencies de 2.

Anàlisi de les dependències

En aquest apartat explicaré una mica les diferents parts del codi, i farem un analisis de dependències per a veure si podem paral·lelitzar o no les diferents parts de l'algoritme..

NOTA: Lo de fons blau és temporal, ho he de consolidar

Funcions auxiliars de comparació

```c
int cmp_fil(const void *pa, const void *pb)
{
tmd * a = (tmd*)pa;
tmd * b = (tmd*)pb;

  if (a->i > b->i) return(1);
  else if (a->i < b->i) return (-1);
  else return (a->j - b->j);
}

int cmp_col(const void *pa, const void *pb)
{
tmd * a = (tmd*)pa;
tmd * b = (tmd*)pb;

  if (a->j > b->j) return(1);
  else if (a->j < b->j) return (-1);
  else return (a->i - b->i);
}
```

En aquestes 2 funcions les usarem per a comparar o bé 2 files o bé 2, les usarem com a suport per a que el quicksort pugui ordenar la matriu dispersa A i la matriu dispersa B.

Aquesta part és esencial ja que les matrius disperses les guardem apollant-nos en un struct que té les coordenades exactes i el valor de la posició (només registrem els no nuls) però inicialment estàn desordenades → per lo que no podem accedir eficientment i hauriem de fer busquedes. Es gracies a aquestos metodes que més endevant podem fer ús de un index de ràpid accés '***jBD***' per exemple.

En cuant a la paral·lelització,no ho he paral·lelitzat ja que no son funcions de suport per a quicksort així que no crec que tingui cap tipus de sentit paral·lelitzar-ho (no se ni si es possible). Així que aquesta part es queda tal cual.

Generació de matrius

```c
for(k=0;k<ND;k++)
    {
        AD[k].i=rand()%(N-1);
        AD[k].j=rand()%(N-1);
        AD[k].v=rand()%100+1;
        while (A[AD[k].i][AD[k].j]) {
            if(AD[k].i < AD[k].j)
                AD[k].i = (AD[k].i + 1)%N;
            else
                AD[k].j = (AD[k].j + 1)%N;
        }
        A[AD[k].i][AD[k].j] = AD[k].v;
    }
    qsort(AD,ND,sizeof(tmd),cmp_fil); // ordenat per files

    for(k=0;k<ND;k++)
    {
        BD[k].i=rand()%(N-1);
        BD[k].j=rand()%(N-1);
        BD[k].v=rand()%100+1;
        while (B[BD[k].i][BD[k].j]) {
            if(BD[k].i < BD[k].j)
                BD[k].i = (BD[k].i + 1)%N;
            else
                BD[k].j = (BD[k].j + 1)%N;
        }
        B[BD[k].i][BD[k].j] = BD[k].v;
    }

    qsort(BD,ND,sizeof(tmd),cmp_col); // ordenat per columnes
```

En aquesta part generem les matrius A i B tant les denses com les disperses. Primer generem els valors aleatoris, evitem les posicions repetides i finalment guardem en la matri densa. La matriu A ordenem els elements per files, per a la matriu B fem lo mateix però ho ordenem per columnes.

Com operem amb randoms, no ho tocaré tampoc, no ho intentaré paralelitzar ja que llavors alterem la llavor i podriem alterar el funcionament del programa.

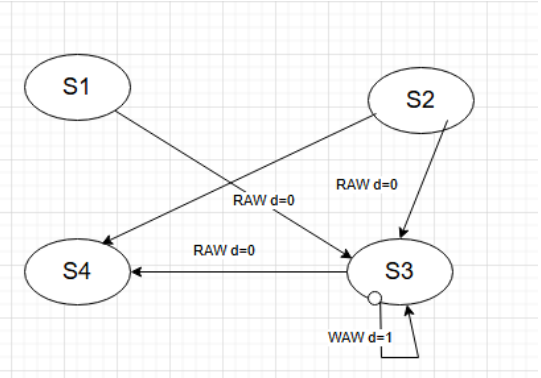
Bucle 1

El bucle 1 lo que fem és inicialitzar l'array de indexos de rapid accés. Aquest "punter" (que no ho és, és més com posar un punt de llibre en BD) és essencial per al funcionament del codi, les matrius disperses les guardem en arrays lineals on només hi guardem els no uls (per estalviar dades, ja que la majoria son els valors nuls els cuals no "participen" en els calculs). Ho fem de la següent manera: BD es un array que representa la matriu dispersa B apojant-nos en el struct custom, cada element de BD te un valor, una columna i una fila (la posició que tindrien a la matriu). Lo que fem llavors aquí es inicialitzar jBD → jBD apunta al primer element de BD que tingui una columna igual o superior a la posició de jBD. Poso un exemple práctic:

- jBD[7]: Això apuntarà al primer element de BD el qual cumpleixi → BD[valor_x].j >= 7. Llavors en jBD[7] = valor_x. Per a saber quin elements té llavors una columna en total, per exemple per a la columna 7 (seguint l'exemple) hauriem de mirar jBD[8] = valor_2. Això ho podem fer perquè previament haviem ordenat BD per columnes (quicksort anterior). Exemple pràctic → si jBD[7] =10 i jBD[8] = 13 llavors tenim que per a la columna 7 hem de mirar, BD[10], BD[11] i BD[12], i llavors podriem tenir per exemple:
    - BD[10].i = 3 || BD[10].j = 7 || BD[10].v = 5
    - BD[11].i = 13 || BD[11].j = 7 || BD[11].v = 8
    - BD[12].i = 25 || BD[12].j = 7 || BD[12].v = 3
- I llavors ja tindriem tots els elements de la columna 7 de la matriu dispersa.

Això com he comentat és importantissim ja que accedir de forma lineal a tots els elements en una matriu dispersa enlloc de poder anar directament als no nuls afectaria molt al rendiment. En quant a l'anàlisi de les dependències he fet lo següent:

```
k=0;                                S1
for (j=0; j<N+1; j++)               S2
{
    while (k < ND && j>BD[k].j)     S3
            k++;                    S4
    jBD[j] = k;                     S5
}
```



Com es pot veure tenim un bucle en S3 i té sentit, ja que té un caracter acumulatiu i per ende es dependent entre iteracions. A més d'això en ningun moment resetejem k , per lo que per així dir-ho, no podriem dividir l'espai entre diferents threads ja que totes les iteracions dependran de les anteriors, tenim condicions de carrera (loop-carried), és per aquest motiu que he decidit deixar aquesta inicialització de forma secuencial.

Bucle 2

En aquest bucle calculem la matriu resultat C1 on multipliquem la matriu dispersa de A (AD) per la matriu densa de B (B). Cada element no nul de A(que tenim gràcies a AD) agafa la fila C de la matriu B i suma a la fila r de C. Exemple si tenim que en AD[k]:

- Si AD[k].i = **2** | AD[k].j = **5** | AD[k].v = **3** llavors fem:
- C1[**2**][i] += **3** * B[**5**][i]. Es a dir agafem la fila **5** de B multiplicada per **3** (el valor de A) a la fila **2** de C1.

```
for(i=0;i<N;i++)
        for (k=0;k<ND;k++)
            C1[AD[k].i][i] += AD[k].v * B[AD[k].j][i];
```

Per aquest "bucle" (que son 2 en realitat) és que he decidit paral·lelitzar el bucle exterior, deixant l'interior de forma secuencial, la idea és que cada thread pugui treballar de forma independent dels altres, repartint-se l'espai de la matriu per files, és per això que que cada thread tindrà la seva propia k i treballarà pel seu compte.

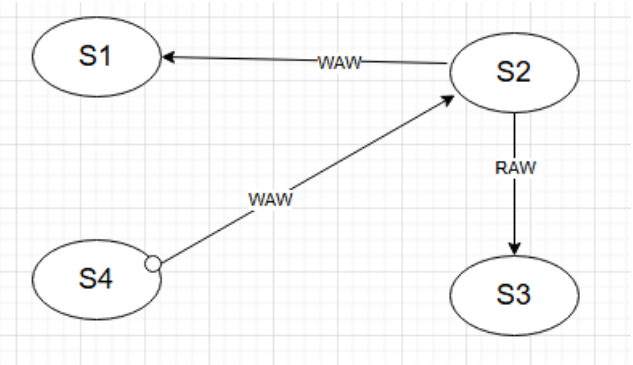
Bucle 3 + 4

En aquesta secció del codi lo que fem és multiplicar la matriu dispersa d'A per la matriu dispera de B (AD * BD) i ho guardem en la matriu de resultat C2. En el bucle 3 (lo que encapsula S1) lo que fem és preparar un vector auxiliar VBcol de mida N. Aquest vector s'usarà com a buffer temporal per emmagatzemar els valors de 1 columna de la matriu dispersa B en format dens. Això ho fem per evitar de recorre repetidament la matriu dispersa BD i poder accedir ràpidament als valors amb l'índex de fila.

Ja després en el bucle 4 fem la multiplicació en si de la matriu dispersa, en el primer for (encapslant S2) lo que fem es inicialitzar VBCol amb els valors de la matriu dispersa de B, i després en el següent for (encapsulant S3) és llavors on apollant-nos en VBCol (i per ende jBD ja que es lo que havíem usat per sabe el rang de BD on trobem els elements de determinada columna) fem l'operació de multiplicació guardant-ho a C2. Després a S4 és on fem la neteja del buffer VBCol per a un futur ús.

```
for (j=0;j<N;j++)
        VBcol[j] = 0;                                   S1

for(i=0;i<N;i++)
{
    for (k=jBD[i];k<jBD[i+1];k++)
          VBcol[BD[k].i] = BD[k].v;                     S2
    for (k=0;k<ND;k++)
         C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j];    S3
    for (j=0;j<N;j++)
          VBcol[j] = 0;                                 S4
}
```



En quant a dependencies podem observar que no hi ha res que sigui problematic per a la paral·lelització, i llavors la meva proposta ah sigut la següent:

```
/* ====== BUCLE 3 ====== */
t0 = omp_get_wtime();
//Matriu dispersa per matriu -> dona matriu Dispersa
#pragma omp threadprivate(VBcol)
for (j=0;j<N;j++){
    VBcol[j] = 0;
}
t_b3 += omp_get_wtime() - t0;

/* ====== BUCLE 4   ====== */
t0 = omp_get_wtime();
#pragma omp parallel for copyin(VBcol)
for(i=0;i<N;i++)
{
    // expandir Columna de B[*][i]
    for (k=jBD[i];k<jBD[i+1];k++)
        VBcol[BD[k].i] = BD[k].v;
    // Calcul de tota una columna de C
    for (k=0;k<ND;k++)
        C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j];
    for (j = 0; j < N; j++)
        VBcol[j] = 0;
}
t_b4 += omp_get_wtime() - t0;
```

Lo que he fet és que el bucle 3 l'he deixat secuencial ja que té un cost computacional molt vaig (l'he mesurat també el temps, i és pràcticament 0), sencillament inicialitzem la copia de VBcol del thread master.

El bucle 4 està paral·lelitzat ja que té una carrega molt major i usem copyin per a que cada thread tingui una copia del array VBcol que haviem inicialitzat en el bucle 3. Això ho faig per a poder tenir una inicialització controlada de VBcol i que tots els threads parteixin desde el mateix punt i estalviar-me inicialitzacions innecesaries.

Bucle 5 +6

En aquesta secció estem fent una multiplicació de matriu dispersa per una altra matriu dispersa. En la primera part inicialitzem a zero les variables auxiliars que utilitzarem :

- neleC: Contador d'elements no nuls de la matriu restant CD.
- VBcol: Buffer temporal per emmagatzemar la coluumna actual de B (igual que abans).
- VCcol: És un buffer temporal per acumular tota una columna de C abans de comprimirla a format dispers.

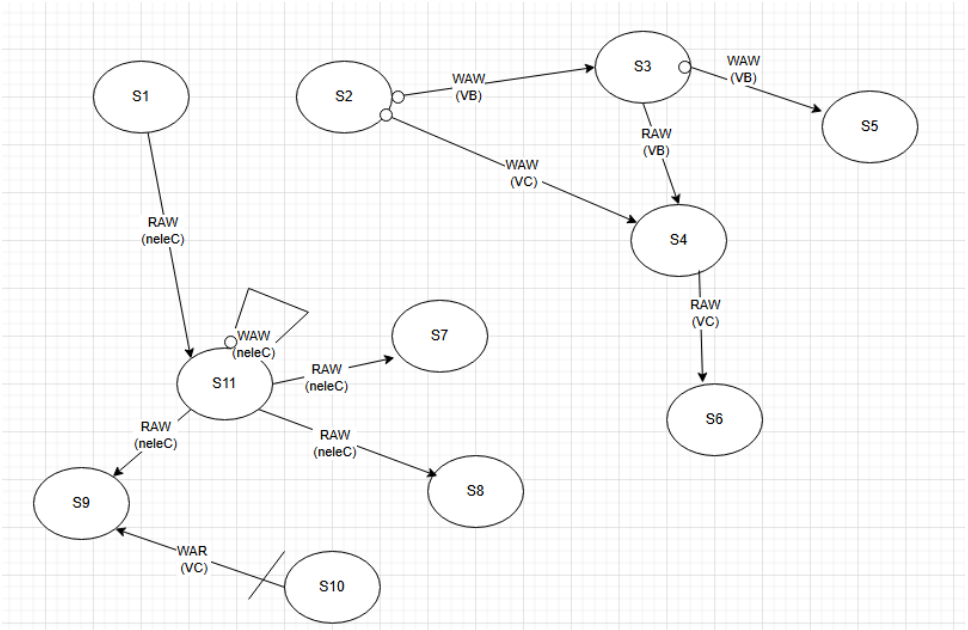
```
// ====== Inicialització ======
neleC=0;                                                S1
for (j=0;j<N;j++)
    VBcol[j] = VCcol[j] = 0;                            S2
// bucle por columnas
for(i=0;i<N;i++)
  {
    // ====== Part 1 ======
    for (k=jBD[i];k<jBD[i+1];k++)
            VBcol[BD[k].i] = BD[k].v;                   S3
    // ====== Part 2 ======
    for (k=0;k<ND;k++)
        VCcol[AD[k].i] += AD[k].v * VBcol[AD[k].j];     S4
    // ====== Part 3 ======
    for (j=0;j<N;j++)
     {
        // neteja vector de B[*][i]
        VBcol[j] = 0;                                   S5
        // Compressio de C
        if (VCcol[j])                                   S6
         {
            CD[neleC].i = j;                            S7
            CD[neleC].j = i;                            S8
            CD[neleC].v = VCcol[j];                     S9
            VCcol[j] = 0;                               S10
            neleC++;                                    S11
         }
     }
  }
```

En la primera part lo que fem es inicialitzar VBcol amb els valors de la amtriu B que corresonen a la columna que estem evaluant la logica es la mateixa que en el cas dels bucles anterior.

En la segona part fem el calcul com a tal, usant VBcol com feiem en la secció anterior, però enlloc de guardar-ho directament ara ho guardem en el vector VCol, això ho fem per a tener una forma densa temporal dels resultats de la coluumna i de la C final.

En la tercera part lo que fem a part de netejar VBcol per a la següent iteració, i guardem en CD els valors no nuls resultants que haviem guardat temporalment en VCol, d'aquesta forma podem tenir la matriu resultant C guardada de forma compresa com haviem fet amb AD i BD, enlloc d'ahver de guardar la matriu sencera amb molts valors nuls.

En quant a les dependències tenim lo següent:



```
neleC=0;
    /* ====== BUCLE 5 ====== */
    #pragma omp threadprivate(VCcol, VBcol)
    for (j=0;j<N;j++)
        VBcol[j] = VCcol[j] = 0;
    /* ====== BUCLE 6 ====== */
    t0 = omp_get_wtime();
    #pragma omp parallel for private(k, j) copyin(VCcol, VBcol)
    for(i=0;i<N;i++)
      {
        for (k=jBD[i];k<jBD[i+1];k++)
                VBcol[BD[k].i] = BD[k].v;
        for (k=0;k<ND;k++)
            VCcol[AD[k].i] += AD[k].v * VBcol[AD[k].j];

        #pragma omp critical
        for (j = 0; j < N; j++)
        {
            // neteja vector de B[*][i]
            VBcol[j] = 0;
            // Compressió de C
            if (VCcol[j]) {
                CD[neleC].i = j;
                    CD[neleC].j = i;
                    CD[neleC].v = VCcol[j];
                    VCcol[j] = 0;
                    neleC++;
            }
        }
      }
      t_b6 += omp_get_wtime() - t0;
```

El bucle 5 el mantenc secuencial degut a que té un cost temporal molt petit (pràcticament despreciable), i inicialitzem els buffers VBcol i VCcol quel els declarem com a threaprivate com haviem fet en el bucle 3 amb VBcol.

En el bucle 6 usem la mateixa tactica que en el bucle 4 i fem que cada thread tingui una copia de VCcol i VBcol que ha sigut inicialitzada en el master thread en el bucle 5. Després designem una secció critica ja que necesitem que la escritura en la solució es faci de forma sincronitzada per assegurar que no hi alterem el resultat respecte el secuencial ja que estem accedint a CD i neleC, cosa que fa que es generi condicions de carrera.

Resultats

Cal comentar una cosa, he observat que en aquest servidor (@zoo.deim.urv.cat) els temps secuencials varien, si executo amb export OMP_NUM_THREADS=1 obtinc els temps secuencials comentats en el moodle, pero si utilitzo -c 1 o no especifico -c x llavors obtinc uns altres temps. Com al manual s'especifica que executem en teen i orca usant -c x, he usat els temps secuencials com a base obtinguts amb aquest metode (amb el codi secuencial original).

El problema es que he intentat llavors provar a usar el export OMP_NUM_THREADS amb la meva versió paral·lela però es capava a un us de 198% de CPU dona igual quants threads estigués assignant, per això he usat el metode que surt a la guia d'execució.

Els temps secuencials obtinguts segons la guia d'execució en teen i orca amb l'arxiu original són:

- En orca: 1:32 -> 92 segons en total.
- En teen: 31 segons.

Els temps que he obtingut amb la meva versió del algorisme seguint la guia d'execució de

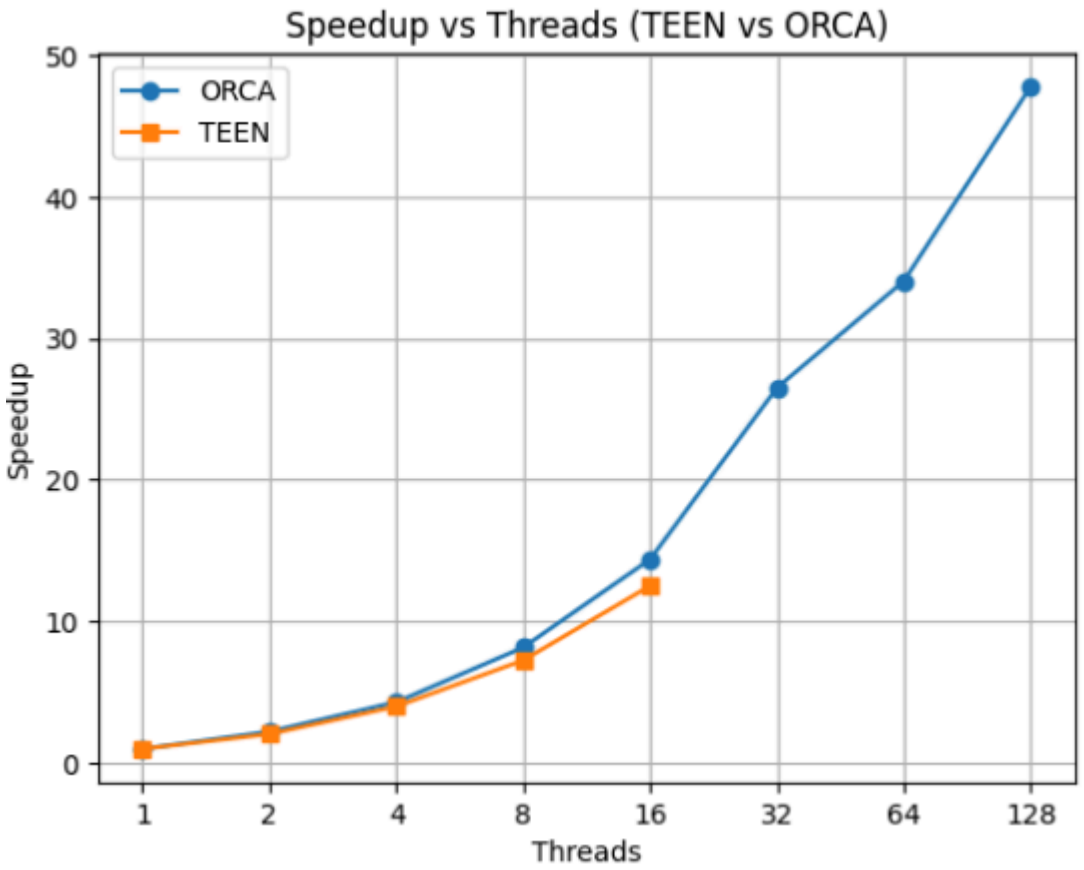
	ORCA							
Threads	1	2	4	8	16	32	64	128
Temps	92,25	41,59	21,43	11,29	6,41	3,48	2,71	1,93

	ORCA							
Threads	1	2	4	8	16	32	64	128
SpeedUp	1	2,22	4,3	8,17	14,39	26,51	34,05	47,79

	TEEN					
Threads	1	2	4	8	16	32
Temps	30,92	15,06	7,73	4,26	2,47	

	TEEN					
Threads	1	2	4	8	16	32
SpeedUp	1	2,05	4	7,26	12,52	

- Mitjana harmonica speedUp ***TEEN***: 4,19.
- Mitjana harmonica speedUp ***ORCA***: 7,27.


Speedup vs Threads (TEEN vs ORCA)
ORCA
TEEN
Speedup
Threads
0
10
20
30
40
50
1
2
4
8
16
32
64
128